

Tolerância a Falhas

Relatório Trabalho Prático

Diogo Marques
PG55931

Francisco Ferreira
PG55942

Ivan Ribeiro
PG55950

Introdução

Neste trabalho prático, pretendemos identificar e analisar possíveis vulnerabilidades do algoritmo Raft perante a falhas assertivas que comprometam as propriedades de segurança do mesmo.

Implementação do Raft

O algoritmo do Raft foi implementado em Rust.

A estrutura principal do código está dividida em módulos que separam as responsabilidades:

- `raft.rs`: Contém a lógica central do algoritmo Raft, incluindo a máquina de estados dos nodos (Seguidor, Candidato, Líder), a replicação de *logs*, e os processos de eleição.
- `maelstrom.rs`: Providencia a abstração para interagir com o ambiente *Maelstrom*, tratando da serialização/desserialização de mensagens e da comunicação entre nodos (ler do *stdin* e escrever para o *stdout*).
- `transport.rs`: Define uma interface de transporte para o Raft (`RaftTransport`) e uma implementação específica para Maelstrom (`MaelstromTransport`), desacoplando a lógica do Raft do mecanismo de transporte subjacente.
- `main.rs`: Inicializa o nodo Maelstrom, integrando o sistema com os testes `lin-kv` do Maelstrom.

Para testar a sua implementação via *Maelstrom* foi utilizado o seguinte comando:

```
cargo build --release
java -jar maelstrom.jar test
  -w lin-kv
  --bin ".\target\release\pessi-raft.exe"
  --time-limit 60
  --node-count 3
  --concurrency 10n
  --rate 100
  --nemesis partition
  --nemesis-interval 3
```

Como resultado, conseguimos observar que o *Maelstrom* não detectou falhas no nosso algoritmo.

> Everything looks good!

No entanto, não é certo que o algoritmo esteja totalmente correto, pois o Maelstrom apenas testa um subconjunto limitado de cenários e não garante a cobertura de todos os possíveis estados ou falhas do sistema.

Verificação de estados com o [Stateright](#)

Para além dos testes de integração com o Maelstrom, recorreu-se ao [Stateright](#) para uma verificação mais formal do algoritmo Raft implementado. O [Stateright](#) é uma ferramenta de *model checking* para sistemas distribuídos escritos em Rust, que permite explorar exaustivamente os possíveis estados de um sistema e verificar se certas propriedades se mantêm, a partir de código.

Funcionamento e Aplicação ao Raft

O *model checking* com [Stateright](#) envolve a definição de:

1. Um **modelo** do sistema: Este modelo descreve os estados possíveis de cada nodo Raft (e.g., `current_term`, `voted_for`, `log`, `commit_index`, `role`) e as ações que podem transitar o sistema de um estado para outro (e.g., enviar/receber uma mensagem `RequestVote`, `AppendEntries`, dar *timeout* numa eleição);
2. Um **ambiente**: Define como as ações dos nodos interagem, incluindo a rede (que pode perder, duplicar ou reordenar mensagens) e outros eventos não determinísticos (mensagens que podem ser enviadas em qualquer momento);
3. **Propriedades**: São invariantes ou condições que devem ser sempre verdadeiras em todos os estados alcançáveis do sistema.

Esta definição foi feita através de:

- implementação do Raft, como modelo;
- ambiente com uma rede que não perde, duplica ou reordena mensagens;
- uma única mensagem, que consiste em escrever o valor 42 na chave 1;¹
- uma lista de propriedades que iremos apresentar a seguir;
- resto dependente do teste a fazer.

Propriedades Verificadas

Foram definidas e verificadas várias propriedades fundamentais do Raft para garantir a sua correção:

1. **Segurança da Eleição (Election Safety)**: No máximo um líder pode ser eleito num determinado termo.
 - Expectativa temporal²: Always
2. **Coerência do Log (Log Safety)**: Se dois *logs* contêm uma entrada *committed* com o mesmo índice e termo, então os *logs* são idênticos até esse índice.
 - Expectativa temporal: Always
3. **Vivacidade do Log (Log Liveness)**: O *log* deverá progredir, isto é, haver um líder que aplique entradas no *log*. O *log* não deverá ficar vazio eventualmente.
 - Expectativa temporal: Sometimes
4. **Vivacidade de Eleições (Election Liveness)**: O sistema deverá progredir de modo a que um líder seja eleito. Um líder deverá ser eleito eventualmente.
 - Expectativa temporal: Sometimes

¹Esta mensagem foi escolhida de forma arbitrária e única para simplificar e acelerar a exploração dos possíveis estados do sistema, sendo suficiente para verificar todas as propriedades.

²Always significa que a propriedade deve ser respeitada em todos os estados alcançáveis do sistema, enquanto que Sometimes significa que a propriedade deve ser respeitada em alguns estados alcançáveis do sistema.

Estas propriedades, que se encontram em `src/fault/property.rs`, foram verificadas com o [Stateright](#) para o algoritmo Raft implementado, com sucesso.

Faltas

Para analisar o comportamento do Raft perante falhas bizantinas e comportamentos maliciosos, criámos uma “API de Faltas” que permite injetar código em pontos críticos da execução. Pontos críticos são por exemplo, numa receção de mensagem, numa transição de estado importante (exemplo: líder eleito), etc. Desta forma, conseguimos simular anomalias sem alterar o código do algoritmo Raft diretamente.

O impacto de cada falha foi avaliado com testes unitários (usando `cargo test`) com a integração com o [Stateright](#), permitindo verificar se as propriedades eram violadas sob essas condições, e se a mitigação de cada falha era eficaz.

Nas secções seguintes, detalhamos as faltas específicas investigadas, o seu impacto potencial no sistema, a demonstração da sua ocorrência, as possíveis medidas de mitigação e a demonstração dessas medidas.

Falta A - Falsificação/Adulteração de Mensagens

Identificação da Falta

Um nodo malicioso pode falsificar ou adulterar (via *man-in-the-middle*) mensagens de outros nodos.

Impacto

O impacto desta falha é extremamente grave, pois um nodo malicioso com capacidade de falsificar ou adulterar mensagens pode comprometer completamente a segurança e a confiabilidade do cluster. Entre os impactos possíveis, destacam-se:

- Derrubar eleições legítimas, falsificando mensagens de votação para impedir a eleição de um líder correto;
- Eleger-se a si próprio, ao manipular votos ou resultados de eleições;
- Injetar entradas falsas no log, comprometendo a integridade dos dados replicados;
- Causar divisões no cluster, enviando mensagens contraditórias para diferentes nodos, levando a estados inconsistentes;
- Impedir a propagação de comandos legítimos, bloqueando ou adulterando mensagens de confirmação;

Em suma, a falsificação de mensagens permite a um atacante assumir controlo total do sistema, violando todas as propriedades de segurança e disponibilidade do protocolo Raft.

Demonstração do Impacto

A demonstração de falsificação via Maelstrom é simples, visto que é possível colocar qualquer “src” na mensagem e o Maelstrom não valida a origem da mensagem. Fora do Maelstrom, num sistema com comunicação via IP, também é possível falsificar mensagens, por exemplo, com recurso a *IP Spoofing*.

No [Stateright](#), que foi onde testámos a falha, conseguimos facilmente encontrar um caso de teste que viola a propriedade de **Election Safety**.

Ao começar uma eleição, o nodo malicioso envia mensagens de votação de todos os nodos para ele próprio, de forma a que ele seja eleito, sem quaisquer verificações de outros nodos.

Desta forma, é possível eleger dois líderes diferentes num mesmo termo, o que viola a propriedade de **Election Safety**.

Localização: src/fault/message_forgery.rs

Ambiente de Teste

• Atores:

- Id(0) — **Malicioso**
- Id(1) — **Seguro**
- Id(2) — **Seguro**

Sequência de Eventos

1. Timeout(Id(1), ElectionTimeout)
2. Id(1) → Raft(VoteRequest(VoteRequestMessage { node_id: Id(1), current_term: 1, log_length: 0, last_log_term: 0 })) → Id(2)
3. Id(2) → Raft(VoteResponse(VoteResponseMessage { node_id: Id(2), current_term: 1, vote_granted: true })) → Id(1)
4. Id(0) → Raft(VoteResponse(VoteResponseMessage { node_id: Id(1), current_term: 1, vote_granted: true })) → Id(0)
5. Timeout(Id(0), ElectionTimeout)
6. Id(0) → Raft(VoteResponse(VoteResponseMessage { node_id: Id(1), current_term: 1, vote_granted: true })) → Id(0)

Quando o nodo malicioso (0) inicia uma eleição, ele simula que recebeu um voto do nodo 1 (basta um para quórum), e assim é eleito como líder, mesmo que já exista outro líder no mesmo termo.

Estado Inválido Encontrado

Node { id: Id(0), current_role: Leader, ... }	Node { id: Id(1), current_role: Leader, ... }	Node { id: Id(2), current_role: Follower, ... }
---	---	---

Propriedades Violadas

- [Segurança da Eleição \(Election Safety\)](#)

Medidas de Mitigação

Seria possível mitigar completamente este problema adicionando autenticação às mensagens, por exemplo, através de assinaturas digitais.

Demonstração das Medidas de Mitigação

Devido à natureza do problema, que não é relacionado com os conteúdos abordados na unidade curricular, não iremos implementar a mitigação.

Falta B - Voto Duplo

Identificação da Falta

Um nodo malicioso pode votar em dois candidatos diferentes numa eleição no mesmo termo.

Impacto

Permite que dois nodos sejam eleitos como líder, o que quebra a propriedade de **Election Safety** do algoritmo.

Demonstração do Impacto

Com o [Stateright](#), podemos injetar código na receção do `VoteRequest` e responder sempre positivamente ao voto, sem verificar se o nodo já votou noutro candidato.

Mais concretamente, o nodo malicioso enviará sempre a mensagem `VoteResponse` com `vote_granted` a `true`, independentemente do voto que recebeu.

Localização: `src/fault/double_vote.rs`

Ambiente de Teste

• Atores:

- `Id(0)` — **Malicioso**
- `Id(1)` — **Seguro**
- `Id(2)` — **Seguro**

Sequência de Eventos

1. `Timeout(Id(0), ElectionTimeout)`
2. `Timeout(Id(2), ElectionTimeout)`
3. `Id(0) → Raft(VoteRequest(VoteRequestMessage { node_id: Id(0), current_term: 1, log_length: 0, last_log_term: 0 })) → Id(1)`
4. `Id(0) → Raft(VoteRequest(VoteRequestMessage { node_id: Id(0), current_term: 1, log_length: 0, last_log_term: 0 })) → Id(2)`
5. `Id(2) → Raft(VoteRequest(VoteRequestMessage { node_id: Id(2), current_term: 1, log_length: 0, last_log_term: 0 })) → Id(0)`
6. `Id(0) → Raft(VoteResponse(VoteResponseMessage { node_id: Id(0), current_term: 1, vote_granted: true })) → Id(2)`
7. `Id(1) → Raft(VoteResponse(VoteResponseMessage { node_id: Id(1), current_term: 1, vote_granted: true })) → Id(0)`

Podemos observar que o nodo malicioso começa a atuar no evento nº6, que deveria de responder ao voto do nodo 2, como `vote_granted: false`, já que tem uma eleição em curso com o `log_length` maior ou igual ao do nodo 2, mas responde `vote_granted: true`. Com o nodo 1 a também responder `vote_granted: true` ao nodo 0, o nodo 0 também é eleito como líder.

Estado Inválido Encontrado

<pre>Node { id: Id(0), current_role: Leader, ... }</pre>	<pre>Node { id: Id(1), current_role: Follower, ... }</pre>	<pre>Node { id: Id(2), current_role: Leader, ... }</pre>
--	--	--

Propriedades Violadas

- [Segurança da Eleição \(Election Safety\)](#)

Medidas de Mitigação

É possível resolver este problema com uma mensagem adicional, por exemplo, `ElectedBy`, que é enviada num fim de eleição, por todos os nodos que foram eleitos para todos os

outros nodos, com a lista de nodos que votaram nele. Caso seja detetado um nodo que votou em dois candidatos diferentes, outros nodos podem ignorar o seu voto futuramente (*black-listed*).

Daqui surge um novo problema, que é, um nodo bizantino pode dizer que recebeu votos de quem não votou nele, o que faria que esse outro nodo fosse *black-listed* indevidamente. Uma solução possível a este problema é recorrer a assinaturas digitais, que permitiriam identificar a autenticidade do voto.

Demonstração das Medidas de Mitigação

Como a segunda parte desta solução não é abordada na unidade curricular, não iremos implementar a mitigação com assinaturas digitais.

No entanto, a primeira parte da solução pode ser implementada, e é o que faremos.

Localização: src/fault/double_vote_fix.rs

Ambiente de Teste

- **Atores:**

- Id(0) — **Malicioso**
- Id(1) — Seguro com mitigação
- Id(2) — Seguro com mitigação

- **Propriedades a verificar:**

- O nodo malicioso é **black-listed**
 - Expectativa temporal: Sometimes

Não conseguimos garantir todas as propriedades de segurança imediatamente após a eleição, porque o nodo malicioso ainda pode causar a existência de dois líderes ao mesmo tempo.

Isto acontece porque a mensagem *ElectedBy*, que permite detetar votos duplicados, só é enviada no final da eleição. Assim, até que essa mensagem seja processada e o nodo malicioso seja identificado e colocado na lista negra (*black-listed*), ainda pode haver uma violação temporária da propriedade de “Election Safety”.

No entanto, após o nodo malicioso ser *black-listed*, é invocada uma eleição imediatamente, e os seus votos passam a ser ignorados nas eleições seguintes, restaurando o funcionamento correto do algoritmo e prevenindo futuras violações desta propriedade.

Apenas precisamos garantir que o nodo malicioso é colocado na lista negra (*black-listed*). As restantes propriedades já foram validadas em cenários sem nodos maliciosos, e, após o *black-list*, o sistema volta a funcionar como se não existissem nodos maliciosos.

Sequência de Eventos

1. Timeout(Id(2), ElectionTimeout)
2. Id(2) → Raft(VoteRequest(VoteRequestMessage { node_id: Id(2), current_term: 1, log_length: 0, last_log_term: 0 })) → Id(0)
3. Timeout(Id(1), ElectionTimeout)

4. $\text{Id}(1) \rightarrow \text{Raft}(\text{VoteRequest}(\text{VoteRequestMessage} \{ \text{node_id: Id}(1), \text{current_term: } 1, \text{log_length: } 0, \text{last_log_term: } 0 \})) \rightarrow \text{Id}(0)$
5. $\text{Id}(2) \rightarrow \text{Raft}(\text{VoteRequest}(\text{VoteRequestMessage} \{ \text{node_id: Id}(2), \text{current_term: } 1, \text{log_length: } 0, \text{last_log_term: } 0 \})) \rightarrow \text{Id}(1)$
6. $\text{Id}(0) \rightarrow \text{Raft}(\text{VoteResponse}(\text{VoteResponseMessage} \{ \text{node_id: Id}(0), \text{current_term: } 1, \text{vote_granted: true} \})) \rightarrow \text{Id}(1)$
7. $\text{Id}(0) \rightarrow \text{Raft}(\text{VoteResponse}(\text{VoteResponseMessage} \{ \text{node_id: Id}(0), \text{current_term: } 1, \text{vote_granted: true} \})) \rightarrow \text{Id}(2)$
8. $\text{Id}(1) \rightarrow \text{Other}(\text{ElectedBy} \{ \text{leader: Id}(1), \text{term: } 1, \text{by: [Id}(1), \text{Id}(0)] \}) \rightarrow \text{Id}(1)$
9. $\text{Id}(2) \rightarrow \text{Other}(\text{ElectedBy} \{ \text{leader: Id}(2), \text{term: } 1, \text{by: [Id}(2), \text{Id}(0)] \}) \rightarrow \text{Id}(1)$

O nodo malicioso (0) viola o protocolo ao conceder voto positivo a dois candidatos diferentes (1 e 2) durante o mesmo termo de eleição. Como resultado, ambos os nodos 1 e 2 acreditam ter obtido a maioria dos votos e assumem simultaneamente o papel de líder, o que viola a propriedade de “Election Safety” do algoritmo Raft. Esta situação só é detetada após o envio das mensagens *ElectedBy*, quando os nodos honestos percebem que o mesmo nodo (0) votou em mais do que um candidato. A partir desse momento, o nodo malicioso é colocado na lista negra (*black-listed*) e os seus votos deixam de ser considerados em eleições futuras, restaurando a segurança do sistema.

Neste caso, apenas o nodo 1 detetou inicialmente o problema; no entanto, em eventos subsequentes, o nodo 2 também irá identificar a infração.

Estado Inválido Encontrado

```
Node {
  id: Id(0),
  blacklist: [],
  current_role: Follower,
  ...
}
```

```
Node {
  id: Id(1),
  blacklist: [Id(0)],
  current_role: Follower,
  ...
}
```

```
Node {
  id: Id(2),
  blacklist: [],
  current_role: Leader,
  ...
}
```

Propriedades Verificadas

- Nodo Malicioso é **black-listed**

Falta C - Negação de Serviço por Spam de Eleições

Identificação da Falta

Cada *follower* tem uma *timer* para dar *timeout* e começar uma eleição, quando não recebe atualizações de um líder. Um nodo malicioso pode simplesmente ignorar esse *timer* e começar sempre uma nova eleição. A cada momento desses, o nodo malicioso, incrementa o seu *term* e tenta eleger-se como líder. Como o seu *term* é maior que o do líder, todos os nodos (incluindo o líder) votam nele, e ele passa a ser o líder.

Este nodo malicioso pode continuar a fazer isto indefinidamente, sempre que um líder é eleito.

Impacto

Como não há um líder estável, a propriedade de *liveness* não é respeitada, isto é, o sistema não consegue fazer progresso, visto que não há tempo suficiente para replicar as entradas no log.

Demonstração do Impacto

Com o [Stateright](#), podemos fazer com que a cada mensagem recebida, o nodo malicioso comece uma eleição.

Localização: src/fault/election_spam.rs

Ambiente de Teste

- **Atores:**

- Id(0) — **Malicioso**
- Id(1) — **Malicioso**
- Id(2) — **Seguro**

Inicialmente, não tínhamos considerado que, com apenas um nodo malicioso, o [Stateright](#) poderia encontrar cenários em que todas as propriedades de segurança fossem satisfeitas. Isto acontece porque, se o nodo malicioso atrasar o envio das mensagens, os outros dois nodos ainda conseguem formar uma maioria e executar o algoritmo Raft corretamente.

Por isso, foi necessário ajustar o teste para incluir dois nodos maliciosos. Assim, garantimos que há sempre eleições a decorrer, impedindo a verificação da propriedade de *Log Liveness*.

Ainda assim, mesmo com dois nodos maliciosos, o [Stateright](#) encontrou uma possibilidade de ainda haver líder na implementação do spam. Se um dos nodos maliciosos começar uma eleição e o nodo 2 responder positivamente, o nodo que começou a eleição passa a ser o líder.

Propriedades Não Verificadas

- [Vivacidade do Log \(Log Liveness\)](#)

Medidas de Mitigação

Para este problema, não há uma solução concreta, e teremos que recorrer a soluções heurísticas.

Uma solução possível é ignorar começos de eleições demasiado cedo, por exemplo, se um *term* começou há menos de 1 minuto, ignorar o começo dele. No entanto, esta solução pode fazer com que o sistema volte demasiado tempo a voltar ao normal caso o líder realmente morra nesse espaço de tempo.

Outra solução possível é limitar o número de eleições que um nodo pode iniciar num dado espaço de tempo, por exemplo, 1 eleição por hora.

Demonstração das Medidas de Mitigação

Optámos por uma abordagem mais simples: se um nodo iniciar eleições em três ocasiões consecutivas, esse nodo é colocado na lista negra (*black-listed*). Desta forma, previne-se que nodos maliciosos possam continuamente desestabilizar o sistema através de spam de eleições.

Localização: src/fault/election_spam_fix.rs

Ambiente de Teste

• Atores:

- Id(0) — **Malicioso**
- Id(1) — Seguro com mitigação
- Id(2) — Seguro com mitigação

• Propriedades a verificar:

- [Segurança da Eleição \(Election Safety\)](#)
- [Vivacidade do Log \(Log Liveness\)](#)
- [Vivacidade de Eleições \(Election Liveness\)](#)
- [Segurança da Eleição \(Election Safety\)](#)
- O nodo malicioso é **black-listed**
 - Expectativa temporal: Sometimes

Todas as propriedades de segurança foram verificadas com sucesso. No entanto, apenas demonstraremos a sequência de eventos que levaram à verificação do nodo malicioso estar na lista negra (*black-listed*).

Sequência de Eventos

1. Timeout(Id(0), ElectionTimeout)
2. Id(0) → Raft(VoteRequest(VoteRequestMessage { node_id: Id(0), current_term: 1, log_length: 0, last_log_term: 0 })) → Id(2)
3. Id(2) → Raft(VoteResponse(VoteResponseMessage { node_id: Id(2), current_term: 1, vote_granted: true })) → Id(0)
4. Id(0) → Raft(VoteRequest(VoteRequestMessage { node_id: Id(0), current_term: 2, log_length: 0, last_log_term: 0 })) → Id(2)
5. Id(2) → Raft(VoteResponse(VoteResponseMessage { node_id: Id(2), current_term: 2, vote_granted: true })) → Id(0)
6. Id(0) → Raft(VoteRequest(VoteRequestMessage { node_id: Id(0), current_term: 3, log_length: 0, last_log_term: 0 })) → Id(2)

Neste cenário, o nodo malicioso (0) inicia três eleições consecutivas, sendo que o nodo 2 recebe todos esses pedidos. Após a terceira tentativa, o nodo 0 é colocado na lista negra (*black-listed*) do nodo 2, impedindo-o de continuar a desestabilizar o sistema.

Estado Válido Encontrado

Node { id: Id(0), blacklist: [], current_role: Candidate, ... }	Node { id: Id(1), blacklist: [], current_role: Follower, ... }	Node { id: Id(2), blacklist: [Id(0)], current_role: Follower, ... }
--	---	--

O resto das propriedades podem ser verificadas rodando o teste unitário que executa o [Stateright](#), presente em src/fault/election_spam_fix.rs.

Propriedades Verificadas

- Nodo Malicioso é **black-listed**

- [Segurança da Eleição \(Election Safety\)](#)
- [Vivacidade de Eleições \(Election Liveness\)](#)
- [Segurança da Eleição \(Election Safety\)](#)
- [Vivacidade do Log \(Log Liveness\)](#)

Falta D - Bifurcação de *log*

Identificação da Falta

Um nodo malicioso pode enviar, no mesmo índice e termo, duas versões diferentes do *log* para *subsets* distintos de nodos. Com isto, diferentes *quorum's* de nodos recebem versões diferentes do *log*, o que faz com que o *log* se bifurque.

Impacto

Se dois conjuntos de nodos aplicarem entradas diferentes no mesmo índice, a propriedade de **Log Safety** é violada: leituras ou escritas podem comportar-se de forma inconsistente em caso de falhas do líder.

Demonstração do Impacto

Para demonstrar essa vulnerabilidade, podemos considerar um cenário em que um nodo malicioso, a cada *LogRequest*, incrementa todos os valores das entradas do *log* em uma unidade antes de enviá-las. Esse comportamento resulta em divergência entre as entradas *committed* dos diferentes nodos, comprometendo a consistência do sistema.

Localização: src/fault/forking_log.rs

Ambiente de Teste

• Atores:

- Id(0) — **Malicioso**
- Id(1) — **Seguro**
- Id(2) — **Seguro**

Sequência de Eventos

1. Timeout(Id(1), ElectionTimeout)
2. Id(1) → Raft(VoteRequest(VoteRequestMessage { node_id: Id(1), current_term: 1, log_length: 0, last_log_term: 0 })) → Id(0)
3. Id(0) → Raft(VoteResponse(VoteResponseMessage { node_id: Id(0), current_term: 1, vote_granted: true })) → Id(1)
4. Id(1) → Raft(LogRequest(LogRequestMessage { leader_id: Id(1), current_term: 1, prefix_length: 0, prefix_term: 0, commit_length: 0, suffix: [] })) → Id(0)
5. Id(0) → Raft(LogResponse(LogResponseMessage { node_id: Id(0), current_term: 1, ack: 0, success: true })) → Id(1)
6. Id(1) → ClientRequest(Write { key: 1, value: 42, msg_id: 1 }) → Id(1)
7. Id(1) → Raft(LogRequest(LogRequestMessage { leader_id: Id(1), current_term: 1, prefix_length: 0, prefix_term: 0, commit_length: 0, suffix: [LogEntry { term: 1, from: Id(1), request: Write { key: 1, value: 42, msg_id: 1 } }] })) → Id(0)
8. Id(0) → Raft(LogResponse(LogResponseMessage { node_id: Id(0), current_term: 1, ack: 1, success: true })) → Id(1)

Observa-se que, no evento nº0, o nodo 1 é eleito como líder. Já no evento nº7, o nodo 0 — que é malicioso — recebe a entrada do log, incrementa o valor e, assim, introduz uma inconsistência no log. Posteriormente, no próximo evento, ao receber a confirmação do nodo 0, o nodo 1 realiza o *commit* dessa entrada adulterada, consolidando a inconsistência no sistema.

Estado Inválido Encontrado

```
Node {
  id: Id(0),
  log: [
    LogEntry {
      term: 1,
      from: Id(1),
      request: Write {
        key: 1,
        value: 43,
        msg_id: 1,
      },
    },
  ],
  commit_length: 0,
  current_role: Follower,
  ...
}

Node {
  id: Id(1),
  log: [
    LogEntry {
      term: 1,
      from: Id(1),
      request: Write {
        key: 1,
        value: 42,
        msg_id: 1,
      },
    },
  ],
  commit_length: 1,
  current_role: Leader,
  ...
}

Node {
  id: Id(2),
  log: [],
  commit_length: 0,
  current_role: Follower,
  ...
}
```

Propriedades Não Verificadas

- [Segurança do Log \(Log Safety\)](#)

Medidas de Mitigação

Uma possível solução para esta falta, seria aquando a receção de um LogRequest contendo novas entradas do *log*, enviar para todos os outros nodos, uma mensagem adicional que contivesse essas novas entradas. Desta forma, os outros nodos veriam que o *log* não é consistente e começariam uma nova eleição, bloqueando o nodo malicioso.

Para evitar grande tráfego de mensagens, podemos aliviar o envio dessa mensagem adicional para apenas alguns nodos e periodicamente, e não a cada LogRequest, como é no protocolo Gossip.

Demonstração das Medidas de Mitigação

A FAZER

Falta E - *Commit* de *log* sem ser aceite pela maioria

Identificação da Falta

Um nodo líder malicioso pode incrementar o *commit_index* sem ter recebido confirmação da maioria dos nodos.

Impacto

Os *logs* podem não estar atualizados numa maioria dos nodos, o que faria com que caso o líder falhasse, dados seriam perdidos. Quebra o princípio de *safety*.

Demonstração do Impacto

Com o [Stateright](#), podemos criar um nodo malicioso que incrementa o `commit_index` para o tamanho do *log* depois de qualquer mensagem ter sido enviada.

Localização: `src/fault/fake_commit_log.rs`

Ambiente de Teste

- **Atores:**

- Id(0) — **Malicioso**
- Id(1) — **Seguro**
- Id(2) — **Seguro**

Neste caso, embora o [Stateright](#) suporte falhas por *crash*, ele não permite que nodos revivam nem simula partições de rede entre os atores. Por esse motivo, não é possível testar cenários em que quando o `commit_index` é incrementado de forma maliciosa, leva a que diferentes nodos tenham entradas *committed* divergentes no *log*, já que não ocorre divergência real de entradas entre os nodos durante a execução.

A EXPLORAR MELHOR

Medidas de Mitigação

Este problema poderia ser resolvido, enviando uma assinatura digital como forma de autenticidade e autenticação em cada `LogResponse`. Assim, o líder era obrigado a guardar essas assinaturas, para as propagar a seguir. Os outros nodos só aceitariam `LogRequest`'s que incrementem o `commit_index`, se nela conter assinaturas digitais válidas de um quorum de nodos.

Demonstração das Medidas de Mitigação

Da mesma forma que na solução da Falta A, devido à natureza do problema, que não está relacionada com os conteúdos abordados na unidade curricular, não iremos implementar a mitigação.